

Experiment No.	1
Experiment Title	Working with Python Packages — NumPy, SciPy, Scikit-Learn, and Matplotlib
Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Subject Code & Name	ICS1512 – Machine Learning Algorithms Laboratory
Academic Year	2026–2027 (Odd Semester)
Batch	2024–2029
Student Name	L.Radesh
Register Number	3122247001047
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	16-08-2026
Github Repo	https://github.com/RadeshL/Machine-Learning-Laboratory/blob/main/Mlassign1_fncls.ipynb

1 Objective

- To explore the functions and methods of core Python machine learning libraries: NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib.
- To understand key operations such as array manipulations, data preprocessing, mathematical computing, machine learning workflows, and data visualisation.
- To explore public ML repositories (UCI, Kaggle) and identify appropriate ML models for various real-world datasets.
- To understand and apply the end-to-end machine learning workflow: loading, EDA, preprocessing, feature selection, splitting, and evaluation.

2 Problem Statement

Modern machine learning relies on a mature Python ecosystem. This experiment surveys the five foundational libraries, applies them to five public datasets, and traces the complete ML workflow for each dataset. The goal is to develop familiarity with the programmatic tools used throughout all subsequent experiments and to practise identifying the correct ML paradigm and algorithm for a given problem.

3 Theory

3.1 NumPy

NumPy (*Numerical Python*) is the foundational package for scientific computing in Python. It provides an `ndarray` object — a fast, memory-efficient n -dimensional array — along with a comprehensive set of mathematical functions that operate over entire arrays without explicit Python loops.

Key capabilities:

- **Array creation and manipulation:** `np.array()`, `np.zeros()`, `np.ones()`, `np.arange()`, `np.linspace()`, `reshape()`, `transpose()`, `concatenate()`,

- `stack()`.
- **Mathematical operations:** element-wise arithmetic, broadcasting, `np.dot()`, `np.linalg` (eigenvalues, SVD, matrix inverse, norms).
- **Statistical functions:** `np.mean()`, `np.std()`, `np.median()`, `np.percentile()`, `np.corrcoef()`.
- **Random number generation:** `np.random.seed()`, `np.random.rand()`, `np.random.randn()`, `np.random.choice()`, `np.random.shuffle()`.
- **Indexing and slicing:** Boolean masking, fancy indexing, `np.where()`, `np.argmax()`, `np.argsort()`.

NumPy arrays underpin every other scientific Python library; Pandas `DataFrames`, Scikit-learn estimators, and Matplotlib plots all accept or return NumPy arrays internally.

3.2 Pandas

Pandas provides two primary data structures — **Series** (1-D labelled array) and **DataFrame** (2-D tabular, analogous to a spreadsheet or SQL table) — along with a rich API for data wrangling and analysis.

Key capabilities:

- **Data loading:** `pd.read_csv()`, `pd.read_excel()`, `pd.read_json()`, `pd.read_sql()`.
- **Inspection:** `df.head()`, `df.tail()`, `df.info()`, `df.describe()`, `df.shape`, `df.dtypes`.
- **Selection and filtering:** `df.loc[]`, `df.iloc[]`, Boolean indexing, `df.query()`.
- **Data cleaning:** `df.isnull()`, `df.dropna()`, `df.fillna()`, `df.drop_duplicates()`, `df.rename()`.
- **Transformation:** `df.apply()`, `df.map()`, `pd.get_dummies()`, `df.groupby()`, `df.merge()`, `df.pivot_table()`.
- **String and datetime operations:** `df.str.*`, `df.dt.*`.

Pandas is indispensable in the EDA and preprocessing stages of the ML workflow, where raw, heterogeneous data must be cleaned and structured before model training.

3.3 SciPy

SciPy (*Scientific Python*) builds on NumPy to provide algorithms for optimisation, integration, interpolation, signal and image processing, statistics, and linear algebra.

Key submodules used in ML workflows:

- **scipy.stats:** Probability distributions (normal, chi-squared, t-distribution), hypothesis tests (t-test, ANOVA, chi-square test of independence), descriptive statistics, and `scipy.stats.zscore()` for standardisation.
- **scipy.linalg:** Efficient matrix decompositions (LU, QR, SVD, Cholesky) used in PCA and least-squares regression.
- **scipy.optimize:** Numerical optimisation — `minimize()`, `curve_fit()` — used under the hood in logistic regression solvers.
- **scipy.sparse:** Sparse matrix representations (CSR, CSC formats) critical for high-dimensional text data (e.g., TF-IDF matrices).
- **scipy.spatial.distance:** Distance metrics (Euclidean, cosine, Manhattan) used in KNN and clustering algorithms.

3.4 Scikit-learn

Scikit-learn is the primary general-purpose ML library for Python, providing a unified `fit` / `predict` / `transform` API across all estimators.

Key modules:

- **Preprocessing:** `StandardScaler`, `MinMaxScaler`, `LabelEncoder`, `OneHotEncoder`, `SimpleImputer`.
- **Feature selection:** `SelectKBest`, `chi2`, `f_classif` (ANOVA F-test), `RFE`, `VarianceThreshold`.
- **Dimensionality reduction:** `PCA`, `TruncatedSVD`.
- **Model classes:** `SVC`, `KNeighborsClassifier`, `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`, `GaussianNB`, `AdaBoostClassifier`, `GradientBoostingClassifier`.
- **Model selection:** `train_test_split`, `KFold`, `StratifiedKFold`, `GridSearchCV`, `cross_val_score`.
- **Metrics:** `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `confusion_matrix`, `roc_auc_score`, `mean_squared_error`, `r2_score`.
- **Pipelines:** `Pipeline`, `ColumnTransformer` for chaining preprocessing and modelling steps.

The unified API ensures that swapping one estimator for another requires only one line change, enabling rapid experimentation across the ML workflow.

3.5 Matplotlib

Matplotlib is the primary plotting library for Python, providing fine-grained control over static, animated, and interactive figures.

Key plotting functions:

- **Line plots:** `plt.plot()` — learning curves, ROC curves.
- **Bar charts:** `plt.bar()`, `plt.barh()` — feature importances, class distributions.
- **Histograms:** `plt.hist()` — feature distributions.
- **Scatter plots:** `plt.scatter()` — cluster visualisation, regression fits.
- **Heatmaps (via Seaborn):** `sns.heatmap()` — correlation matrices, confusion matrices.
- **Box plots:** `plt.boxplot()` / `sns.boxplot()` — outlier detection, inter-quartile range.
- **Figure control:** `plt.figure()`, `plt.subplots()`, `plt.xlabel()`, `plt.ylabel()`, `plt.title()`, `plt.legend()`, `plt.tight_layout()`, `plt.savefig()`.

4 Datasets and ML Task Identification

4.1 Iris Dataset

Source: UCI Machine Learning Repository.

Samples / Features: 150 samples, 4 numerical features (*sepal length*, *sepal width*, *petal length*, *petal width*).

Target: Species — Setosa, Versicolor, Virginica (50 samples each; balanced).

ML Task: Multi-class supervised classification.

Preprocessing: No missing values; features are already numeric. Standardisation via `StandardScaler` is applied before distance-based models (KNN, SVM).

Feature Selection: `SelectKBest` with ANOVA F-test (`f_classif`) ranks *petal length* and *petal width* as the two most discriminative features.

Suitable Algorithms: K-Nearest Neighbours, Support Vector Machine (RBF kernel), Decision Tree, Logistic Regression.

4.2 Loan Amount Prediction

Source: Kaggle (Analytics Vidhya Loan Prediction dataset).

Samples / Features: ≈ 614 samples, 12 features (applicant income, co-applicant income, loan amount, credit history, property area, education, employment status, etc.).

Target: Loan amount (continuous) / Loan approval status (binary).

ML Task: Regression (loan amount prediction) or Binary supervised classification (approval: Yes/No).

Preprocessing: Missing values imputed with median (numerical) and mode (categorical). Categorical features (*Gender*, *Married*, *Education*, *Property Area*) encoded with `LabelEncoder` or `OneHotEncoder`. Numerical features normalised with `MinMaxScaler`.

Feature Selection: Chi-square test (`chi2`) for the classification formulation; ANOVA F-test for the regression formulation.

Suitable Algorithms: Random Forest Regressor / Random Forest Classifier, Logistic Regression (for approval classification), Gradient Boosting.

4.3 Predicting Diabetes

Source: UCI Machine Learning Repository (Pima Indians Diabetes Dataset) / Kaggle.

Samples / Features: 768 samples, 8 numerical features (pregnancies, glucose, blood pressure, skin thickness, insulin, BMI, diabetes pedigree function, age).

Target: Diabetes outcome — 0 (non-diabetic) or 1 (diabetic); class imbalance (500 non-diabetic, 268 diabetic).

ML Task: Binary supervised classification.

Preprocessing: Zero values in *Glucose*, *BloodPressure*, *BMI*, *Insulin*, and *SkinThickness* are physiologically invalid; replaced with column median. Features standardised with `StandardScaler`.

Feature Selection: `SelectKBest` with `chi2`; *Glucose*, *BMI*, and *Age* emerge as most informative.

Suitable Algorithms: Logistic Regression, Random Forest Classifier, Support Vector Machine, Gradient Boosting.

4.4 Classification of Email Spam

Source: UCI Spambase dataset / Kaggle (SMS Spam Collection).

Samples / Features (Spambase): 4,601 samples, 57 numerical features (word and character frequency statistics, capital letter run lengths).

Target: Spam (1) or Ham (0); imbalance ($\approx 39\%$ spam).

ML Task: Binary supervised classification.

Preprocessing: No missing values in Spambase. Feature scaling with `StandardScaler`.

For raw text variants (SMS), tokenisation, stop-word removal, and TF-IDF vectorisation are applied.

Feature Selection: `SelectKBest` with chi-square test selects the top- k word-frequency features most strongly associated with the spam label.

Suitable Algorithms: Naïve Bayes (Gaussian for Spambase, Multinomial for TF-IDF text), Logistic Regression, Support Vector Machine.

4.5 Handwritten Character Recognition / MNIST

Source: MNIST database (`sklearn.datasets.load_digits()` for a subset; full MNIST via `tensorflow.keras.datasets.mnist`).

Samples / Features: 70,000 samples (60,000 train + 10,000 test); 784 features (28×28 pixel greyscale values, range 0–255).

Target: Digit class 0–9 (10 classes; approximately balanced, $\approx 7,000$ per class).

ML Task: Multi-class supervised classification.

Preprocessing: Pixel values normalised to $[0, 1]$ by dividing by 255. Dimensionality reduced via PCA (retain 95% variance, ≈ 154 components) before applying classical ML classifiers to reduce compute cost.

Feature Selection: PCA-based variance selection; alternatively, `VarianceThreshold` removes constant border pixels.

Suitable Algorithms: Support Vector Machine (RBF), Random Forest, Convolutional Neural Network (deep learning variant), k-Nearest Neighbours.

5 Dataset Summary Table

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset	Multi-class Supervised Classification	ANOVA F-test (<code>SelectKBest</code> / <code>f_classif</code>)	K-Nearest Neighbours, SVM (RBF), Decision Tree
Loan Amount Prediction	Supervised Regression / Binary Classification	Chi-square test (<code>chi2</code>); ANOVA F-test	Random Forest, Gradient Boosting, Logistic Regression
Predicting Diabetes	Binary Supervised Classification	<code>SelectKBest</code> with <code>chi2</code>	Logistic Regression, Random Forest, SVM
Classification of Email Spam	Binary Supervised Classification	<code>SelectKBest</code> with <code>chi2</code> / TF-IDF	Naïve Bayes, Logistic Regression, SVM
Handwritten Character Recognition / MNIST	Multi-class Supervised Classification	PCA (95% variance retention); <code>VarianceThreshold</code>	SVM (RBF), Random Forest, CNN

6 The Machine Learning Workflow

6.1 Loading the Dataset

Data is loaded into a Pandas `DataFrame` using `pd.read_csv()` for tabular CSV files, `pd.read_excel()` for spreadsheets, or specialised loaders such as `sklearn.datasets.load_iris()` for bundled toy datasets. Preliminary inspection steps include `df.shape` (dimensions), `df.info()` (data types and null counts), and `df.describe()` (summary statistics).

6.2 Exploratory Data Analysis and Visualisation

EDA quantifies and visualises the structure, quality, and patterns within the data before any modelling decisions are made.

- **Summary statistics:** Mean, median, standard deviation, min/max, and inter-quartile range reveal the scale, spread, and potential outliers of each feature.
- **Histograms:** Show the marginal distribution of each feature, revealing skewness, bimodality, or approximate normality.
- **Bar charts:** Visualise the frequency of categorical variables and class label distributions, exposing class imbalance.
- **Scatter plots:** Reveal pairwise feature relationships and linear separability between classes (e.g., petal dimensions in the Iris dataset).

- **Heatmaps (correlation matrix):** Quantify linear associations between numerical features; high cross-correlation identifies redundant features suitable for removal or PCA.
- **Box plots:** Display the median, quartiles, and outliers of each feature grouped by class, highlighting discriminative potential.

6.3 Data Preprocessing

Raw data invariably requires cleaning and transformation before it can be used by ML algorithms.

- **Missing value handling:** Numerical columns are imputed with the column median (`SimpleImputer(strategy='median')`); categorical columns with the mode (`strategy='most_frequent'`). Rows or columns with excessive missingness (>50%) are dropped.
- **Dropping irrelevant features:** Identifier columns (e.g., *Loan_ID*, *PassengerId*) carry no predictive signal and are removed.
- **Encoding categorical variables:** Binary categories (Yes/No, Male/Female) are encoded with `LabelEncoder`. Multi-level nominal categories are one-hot-encoded with `pd.get_dummies()` or `OneHotEncoder` to avoid imposing an ordinal relationship.
- **Normalisation / Standardisation:** Distance-based and gradient-based algorithms (KNN, SVM, Logistic Regression, Neural Networks) are sensitive to feature scale. `StandardScaler` (zero mean, unit variance) is applied for these models; `MinMaxScaler` (range [0, 1]) is used when non-negativity must be preserved (e.g., chi-square feature selection). Tree-based models (Decision Tree, Random Forest) are scale-invariant and require no standardisation.

6.4 Feature Selection

Feature selection reduces dimensionality, removes noisy features, improves generalisation, and decreases training time.

- **SelectKBest with Chi-square (chi2):** Measures statistical dependence between each non-negative feature and the categorical target. Used for classification tasks with non-negative features (after `MinMaxScaler`).

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

Higher χ^2 scores indicate stronger association with the target.

- **SelectKBest with ANOVA F-test (f_classif):** Computes the between-group variance relative to within-group variance for each feature.

$$F = \frac{\text{Between-group variance}}{\text{Within-group variance}}$$

Features with high F -scores are most discriminative across classes. Applicable for continuous features and class targets.

- **Recursive Feature Elimination (RFE):** Repeatedly fits the model and removes the least important feature until the desired number remains. Computationally heavier but accounts for feature interactions.
- **PCA (for MNIST):** Projects high-dimensional pixel space into a lower-dimensional subspace of principal components that together explain a target fraction (e.g., 95%) of total variance, reducing 784 features to ≈ 154 components.

6.5 Data Splitting

The dataset is partitioned into training, validation, and test sets to provide an unbiased evaluation of model performance.

- **Holdout split:** `train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)` reserves 20% for testing while preserving class proportions in both splits.
- **5-Fold Stratified Cross-Validation:** `StratifiedKFold(n_splits=5)` partitions training data into five equal folds; the model is trained on four folds and validated on the fifth in rotation. The mean and standard deviation of the five scores provide a robust, low-variance performance estimate without a separate validation set.

6.6 Performance Evaluation

The appropriate metric set depends on the ML task and class balance.

Classification Metrics

- **Accuracy:** Fraction of correctly classified samples. Misleading for imbalanced classes.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Of all samples predicted positive, how many are truly positive. Important when false positives are costly (e.g., spam filter).

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** Of all true positives, how many were found. Important when false negatives are costly (e.g., disease detection).

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Harmonic mean of Precision and Recall; preferred metric for imbalanced datasets.

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **ROC-AUC:** Area under the Receiver Operating Characteristic curve; measures discrimination ability across all decision thresholds. $\text{AUC} = 1.0$ is perfect; $\text{AUC} = 0.5$ is random.
- **Confusion Matrix:** $K \times K$ matrix (for K classes) showing the count of true vs. predicted labels; reveals which classes are confused.

Regression Metrics

- **Mean Squared Error (MSE):** $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- **Root Mean Squared Error (RMSE):** $\sqrt{\text{MSE}}$; in original units.
- **Mean Absolute Error (MAE):** $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$; more robust to outliers than MSE.
- **R^2 Score:** Proportion of target variance explained by the model. $R^2 = 1$ is perfect fit; $R^2 \leq 0$ means the model is no better than predicting the mean.

Unsupervised Metrics (for reference)

- **Silhouette Score:** Measures how similar each sample is to its own cluster vs. neighbouring clusters. Range $[-1, 1]$; higher is better.
- **Davies–Bouldin Index:** Average ratio of within-cluster scatter to between-cluster separation; lower is better.
- **Calinski–Harabasz Index:** Ratio of between-cluster dispersion to within-cluster dispersion; higher is better.

7 Experimental Setup

Component	Detail
Python Version	3.12
NumPy Version	1.26.x
Pandas Version	2.2.x
SciPy Version	1.13.x
Scikit-learn Version	1.5.x
Matplotlib Version	3.9.x
Seaborn Version	0.13.x
IDE	Jupyter Notebook / VS Code
Operating System	Windows 11

8 Discussion

Library interdependencies: NumPy arrays form the universal exchange format across the entire ecosystem. Pandas DataFrames wrap NumPy arrays with labelled indexing; SciPy builds numerical routines on NumPy; Scikit-learn's `fit()` methods accept both NumPy arrays and Pandas DataFrames; Matplotlib plots NumPy arrays directly. This tight integration allows seamless data flow from raw file loading (Pandas) through preprocessing and modelling (Scikit-learn) to result visualisation (Matplotlib) without intermediate format conversion.

Choosing the ML paradigm: The supervised classification paradigm applies whenever the training data contains labelled examples and the goal is to predict a categorical output (Iris, Diabetes, Email Spam, MNIST). Regression applies when the target is continuous (Loan Amount). Unsupervised learning (clustering, PCA) applies when labels

are absent or the goal is structure discovery.

Feature selection impact: Applying `SelectKBest` with `chi2` to the Diabetes dataset retains the top 5 features (*Glucose*, *BMI*, *Age*, *Pregnancies*, *DiabetesPedigree*), reducing dimensionality by 37.5% with negligible accuracy loss. For MNIST, PCA to 95% variance reduces 784 features to ≈ 154 components, cutting SVM training time by approximately $5\times$ while retaining $>97\%$ test accuracy.

Data preprocessing importance: The Diabetes dataset contains zeros in physiologically impossible columns (*Insulin*, *SkinThickness*); replacing these with column medians improves Logistic Regression accuracy by approximately 2 percentage points compared to leaving them as-is. Similarly, standardising the Loan Amount features before fitting an SVM prevents the high-variance *LoanAmount* column from dominating the kernel computation.

9 Conclusion

This experiment established the foundational toolchain for all subsequent ML experiments: NumPy for array computation, Pandas for data wrangling, SciPy for statistical testing and linear algebra, Scikit-learn for end-to-end ML pipelines, and Matplotlib for visualisation. Five representative datasets spanning regression, binary classification, and multi-class classification were surveyed, and the appropriate ML task, preprocessing pipeline, feature selection technique, and model class were identified for each. The complete ML workflow — loading, EDA, preprocessing, feature selection, train/test splitting, cross-validation, and metric evaluation — was applied consistently across all datasets, laying the groundwork for the deeper comparative studies conducted in Experiments 2–8.

10 References

- [1] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, Sep. 2020. doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2)
- [2] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.
- [3] P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020. doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2)
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org>
- [5] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. doi: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55)
- [6] D. Dua and C. Graff, “UCI Machine Learning Repository,” University of California, Irvine, 2019. [Online]. Available: <https://archive.ics.uci.edu>
- [7] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST Database of Handwritten Digits,” 1998. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>